



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 9. Оптимизация затрат на ИИ. Стратегии развертывания ИИ-моделей (/teach/control/stream/view/id/931478150)

РЕ9.1 Мульти-модельный ИИ для эффективной работы с ресурсами

Следующий урок (/pl/teach/control/lesson/view?id=338667561)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Уважаемые студенты!

В рамках данного урока могут рассматриваться примеры передачи персональных данных при регистрации на иностранных сервисах, а также обработки и хранения пользовательских данных.

В соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных», обращаем ваше внимание: в демонстрационных примерах и при выполнении домашних

заданий необходимо использовать исключительно тестовые данные или тестовые аккаунты, в которых не указаны персональные данные — используйте вымышленные ники, дополнительные данные и т.п.

Использование иностранных сервисов не является обязательным. Вы вправе выполнять домашние задания на аналогичных разрешённых платформах по вашему выбору.

Результат дня

Научишься создавать мультимодельную систему для оптимизации запросов по токенам.

Теория на сегодня

- токенизация — как работает и как оценить число токенов;
- оптимизация запросов по токенам;
- мультимодельные системы для оптимизации затрат.

Время изучения материалов

 Время чтения и просмотра: ~ 50 мин

 Время выполнения задания: ~ 1 час

1. Токенизация

Эффективная работа с ресурсами

Вы остановились на 04:51

ПРОДОЛЖИТЬ

СНАЧАЛА

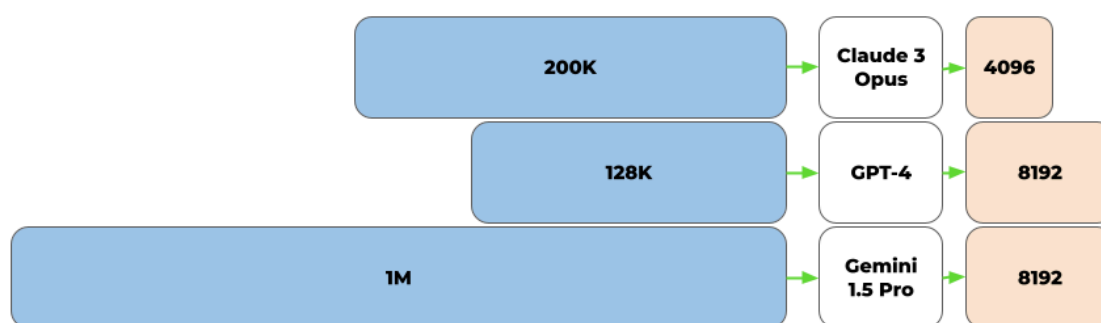
И как
ОПТИМИ

zerocoder.ru



В этом уроке ты узнаешь, как эффективно работать с ресурсами, как происходит токенизация в больших языковых моделях и как мультимодельный ИИ помогает оптимизировать затраты.

При сравнении моделей ИИ (Claude, ChatGPT, Gemini) важным был такой параметр, как **контекстное окно**. Каждая модель может обрабатывать последовательность определённой длины. Длина измеряется в токенах. Чтобы понять, как модель работает с информацией, нужно понимать, как модель разбивает информацию на токены.



Языковые модели имеют **ограниченное количество токенов**, которые они могут сгенерировать, то есть они не способны создавать бесконечно длинные тексты. **Это ограничение связано с вычислительными и экономическими аспектами их работы.** Каждая модель имеет свою стоимость использования, и эта стоимость выражается в количестве токенов. Чем больше токенов обрабатывает и генерирует модель, тем больше стоит её использование. Поэтому важно понимать, как работает токенизация и как можно её оптимизировать, чтобы сократить расходы.

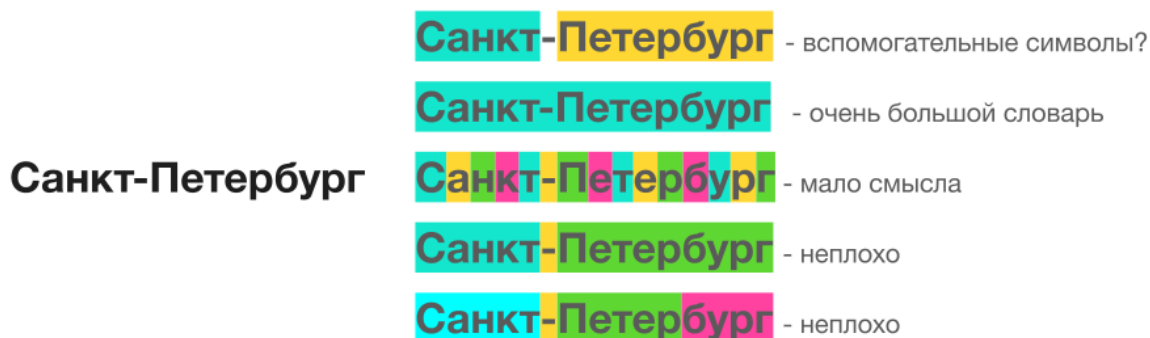


Токен — это базовая единица данных, используемая для обработки информации.

Рассмотрим на примере слово "Санкт-Петербург". Токенизация этого слова может происходить по-разному в зависимости от задачи:

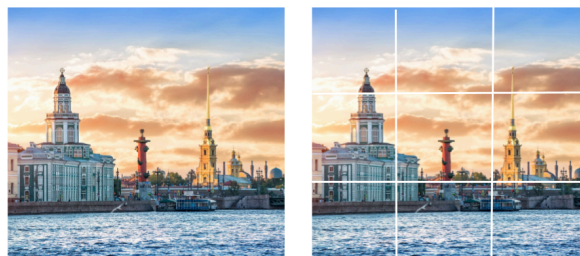
- **классификация текста:** можно выбросить вспомогательные символы (дефисы) и разбить слово на два токена: "Санкт" и "Петербург";
- **простое деление по пробелам:** можно рассматривать "Санкт-Петербург" как один токен. Однако это приведет к увеличению словаря, что усложнит работу модели и снизит её производительность.
- **побуквенная токенизация:** каждая буква и каждый дефис будут отдельными токенами. Это уменьшает размер словаря, но делает систему менее эффективной, так как отдельные буквы несут меньше смысла по сравнению с целыми словами или словосочетаниями. Также при такой токенизации контекст будет сложно сделать объёмным: 128 тысяч токенов смогут вместить только 128 тысяч букв.
- **деление слов на части:** современные модели используют компромиссный подход: они делят слова на части, которые часто встречаются в разных контекстах. Например, "Санкт-Петербург" можно разбить на три токена: "Санкт", "-", "Петербург", или на четыре: "Санкт", "-", "Петер", "бург".

Такое деление помогает модели эффективно работать с частями слов, которые имеют самостоятельное значение, как в примерах с "Екатеринбург", "Оренбург" и "Зальцбург".



Токенизируется не только текст.

Картинки, видео и аудио также подвергаются токенизации. Например, изображения разбиваются на несколько патчей, которые затем обрабатываются моделью. Так как все современные модели основаны на архитектуре трансформеров, они работают с токенами независимо от типа данных.



При работе с изображениями **сложно предсказать точное количество токенов**, так как некоторые модели, например, модели от OpenAI, могут описывать картинку текстом и оценивать стоимость использования не по количеству токенов, а по медиафайлу.

Gemini, например, использует около 250 токенов на изображение.

Токенизация на разных языках

На разных языках и алфавитах токенизация может работать по-разному, и это можно протестировать, введя разные тексты на сайте platform.openai.com/tokenizer (<http://platform.openai.com/tokenizer>)

Например, на английском языке один токен соответствует одному слову (при наличии апострофа — два токена), на русском языке один токен соответствует сочетанию букв.

Токенизация различается и у моделей. Более старые модели ChatGPT могли воспринимать буквы русского алфавита как один или два токена.

Проблемы и ограничения токенизации

- чем больше токенов используется, тем выше стоимость;
- чем больше токенов занимает текст, тем меньше информации можно поместить в контекст модели;
- чем больше токенов, тем дольше модель их обрабатывает, что может замедлить работу;
- большое количество токенов усложняет извлечение нужных данных из контекста.

✂ Токенизация в закрытых моделях, таких как те, что доступны через API, **не поддается изменениям со стороны пользователя**. Поэтому важно оптимизировать количество токенов со своей стороны, чтобы снизить расходы и повысить эффективность работы модели.

2. Оптимизация запросов



Вы остановились на 09:37

- Промт
- Стоп-с
- Ограни

ПРОДОЛЖИТЬ

СНАЧАЛА



zerocoder.ru

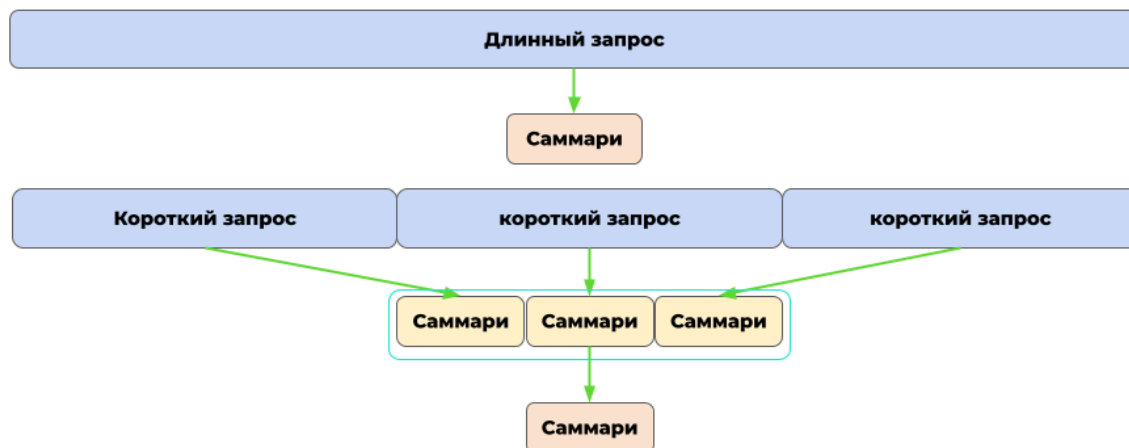
При работе с языковыми моделями важно понимать, как оптимизировать запросы, чтобы эффективно использовать доступные ресурсы. Ниже приведены основные стратегии работы с длинными промтами и ответами моделей.

1. **Сокращение преамбул:** Модели часто добавляют преамбулу перед основным ответом. Например, при запросе сочинить стих в стиле Пушкина модель может начать с "Да, конечно, нет проблем" и далее добавить подводку. Эту часть можно сократить, задав правильный промт, например: "Отвечай коротко, без преамбулы".
2. **Использование стоп-слов:** Можно задать набор стоп-слов, после генерации которых модель автоматически останавливается. Это позволяет контролировать длину ответа.
3. **Ограничение длины генерации:** В API моделей можно задать ограничение на длину генерируемого текста. Это позволяет контролировать объем ответа и избежать перерасхода токенов.

Работа с длинными входами требует особого подхода. Вот несколько стратегий, которые помогут справиться с длинными запросами:

1. **Стратегия "разделяй и властвуй":** Очень длинный запрос можно разбить на несколько коротких частей. Например, разбить книгу на главы и обработать каждую главу отдельно. Затем получить

короткое резюме каждой главы и объединить их для создания общего резюме всего документа. Такой метод называется chunking



2. Использование более простых моделей: Разделение текста на короткие части позволяет использовать более простые и дешевые модели с меньшим контекстом, например, 64,000 токенов вместо более крупных моделей.

3. Фильтрация запросов: Важно удалить всю лишнюю информацию из запросов. Например, при парсинге веб-страниц можно использовать библиотеку BeautifulSoup для извлечения только текстового содержания, без HTML-тегов, что значительно сократит количество токенов.

4. Адаптация форматов данных: При работе с структурированными данными, такими как JSON, можно удалить лишние символы, такие как кавычки, двоеточия и пустые значения. Это позволит сократить количество токенов, сохраняя при этом всю важную информацию.

```
{
  "name": "Иван Петров",
  "age": 75,
  "spouse": { "name": "Ольга Петрова", "age": 72, "married_since": "1970-05-20"},
  "children": [
    { "name": "Алексей Иванов",
      "age": 50,
      "spouse": { "name": "Екатерина Иванова", "age": 48, "married_since": "1995-09-10"},
      "children": [
        { "name": "Дмитрий Алексеев", "age": 25, "spouse": null, "children": [] },
        { "name": "Анна Алексеева", "age": 22, "spouse": null, "children": [] }
      ]
    },
    {
      "name": "Наталья Сидорова",
      "age": 48,
      "spouse": { "name": "Александр Сидоров", "age": 50, "married_since": "1998-07-15"},
      "children": [
        { "name": "Елена Сидорова", "age": 20, "spouse": null, "children": [] }
      ]
    }
  ]
}
```

287 токенов

```
{
  [Иван Петров, 75, [Ольга Петрова, 72, 1970-05-20],
  children : [
    [Алексей Иванов, 50, [Екатерина Иванова, 48, 1995-09-10],
    children: [
      [Дмитрий Алексеев, 25],
      [Анна Алексеева, 22]
    ]
  ],
  [Наталья Сидорова, 48, [Александр Сидоров, 50, 1998-07-15],
  children : [
    [Елена Сидорова, 20]
  ]
  ]
}
```

178 токенов

-37%

Низкоресурсные языки

При работе с низкоресурсными языками остро стоит проблема токенизации запросов.

✚ Тексты на языке с меньшим количеством обучающих данных разбиваются на большее количество токенов.

Это касается даже языков, использующих одинаковый алфавит.

Например, слова на русском и белорусском языках будут токенизироваться по-разному, так как некоторые комбинации букв встречаются реже в обучающей выборке модели. Например, сочетание "ш" и "и" в русском языке объединяется в один токен, так как встречается чаще, чем в белорусском, а белорусское "шы" будет разбито на два токена, так как не встречается в русском языке.

ШИПОВНИК

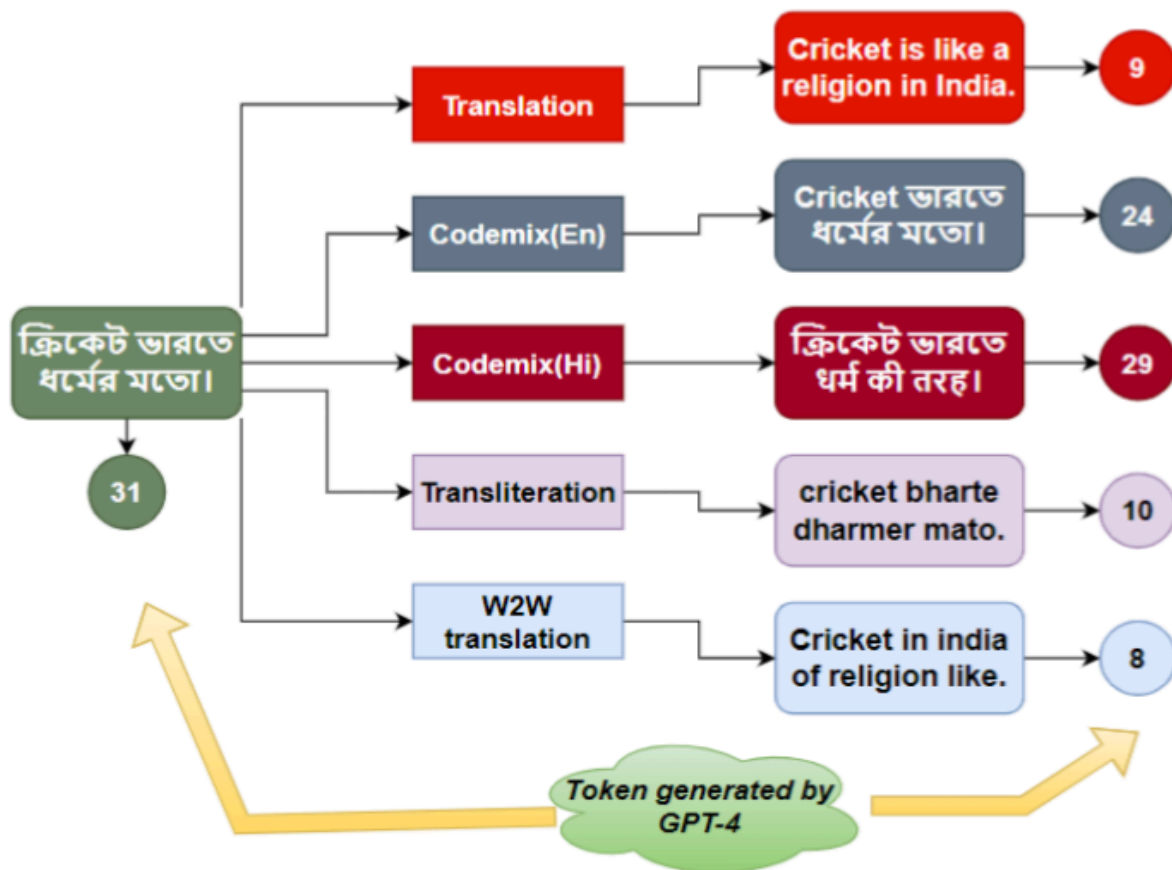
ШЫПШЫНА

Ситуация усложняется при работе с языками, использующими разные или похожие алфавиты, но с отличными символами. Например, в белорусском языке есть буквы, отсутствующие в русском, в болгарском и чешском языках также есть уникальные буквы. Чем меньше данных по данному языку в обучающей выборке модели, тем хуже модель будет с ним работать и тем больше токенов потребуется для его обработки.

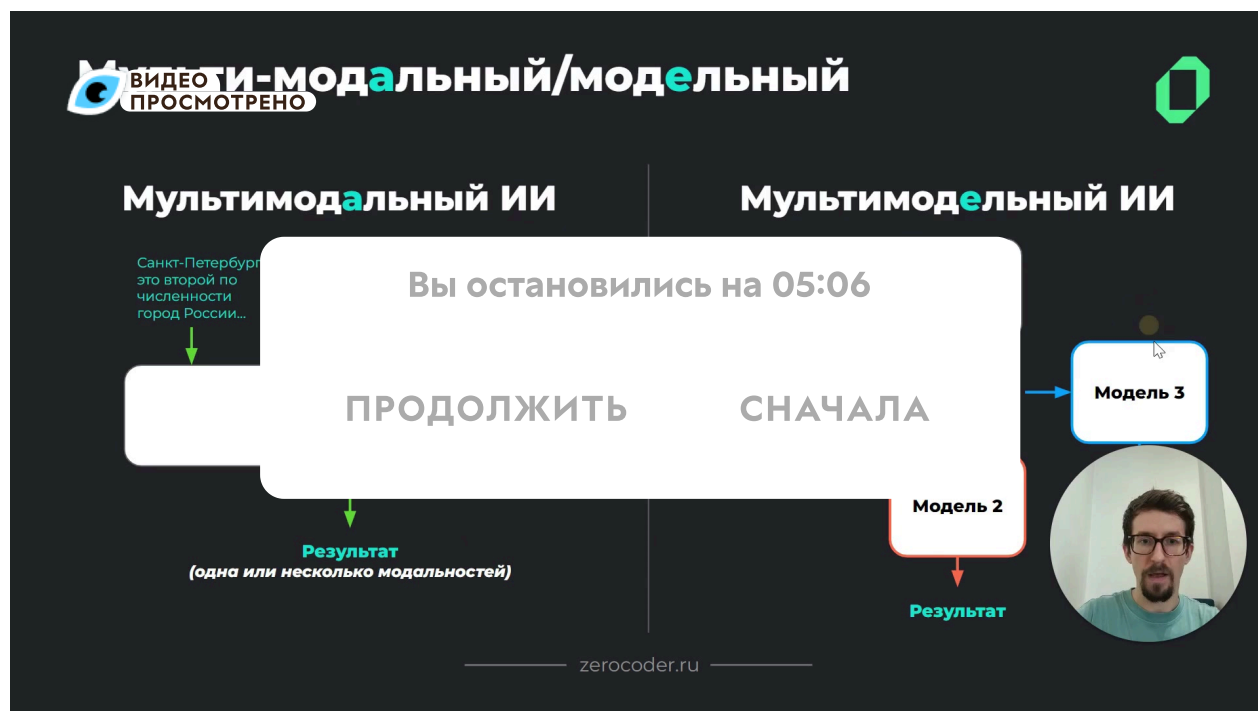
Методы оптимизации токенизации

1. **Перевод на более ресурсный язык:** Один из самых простых подходов — перевод текста на язык с большим количеством обучающих данных, например, на английский. При работе с русскоязычными моделями, такими как GigaChat или Yandex GPT, можно переводить на русский язык. Для этого понадобится доступ к дешевой или локальной модели, выполняющей перевод.
2. **Транслитерация:** Перевод текста на более популярный алфавит, например, запись слов английскими буквами. Это позволяет сократить количество токенов и обрабатывать тексты большей длины. Хотя при транслитерации возможно некоторое снижение качества, этот подход остается приемлемым. (Не "привет", а "privet").
3. **Имплицитный перевод:** В промпте можно указать модели, что для лучшего понимания текста она может мысленно перевести его на английский. Это не уменьшит количество токенов, но улучшит качество обработки текста, так как модель будет лучше понимать контекст.

 For your better understanding you can translate the sentence to English in background when needed.



3. Мультимодальный искусственный интеллект



Мультимодальный и мультимодельный искусственный интеллект

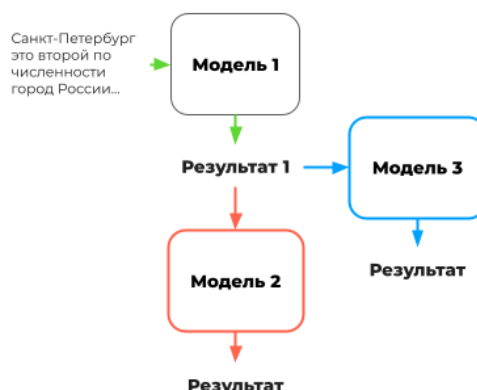
👉 **Мультимодальный искусственный интеллект** — это ИИ, который может работать с данными разных типов (тексты, картинки, аудио, видео и другие). Он принимает данные различных модальностей и генерирует результаты в одной или нескольких из них.

👉 **Мультимодельный искусственный интеллект** — это система, состоящая из нескольких моделей. Например, одна модель обрабатывает запрос, и на основе промежуточного результата выбирается следующая модель для дальнейшей обработки. Примеры таких систем включают комбинацию моделей, как в случае с ChatGPT, где используется GPT для работы с текстами, CLIP для понимания изображений и DALL-E для генерации картинок.

Мультимодальный ИИ



Мультимодельный ИИ

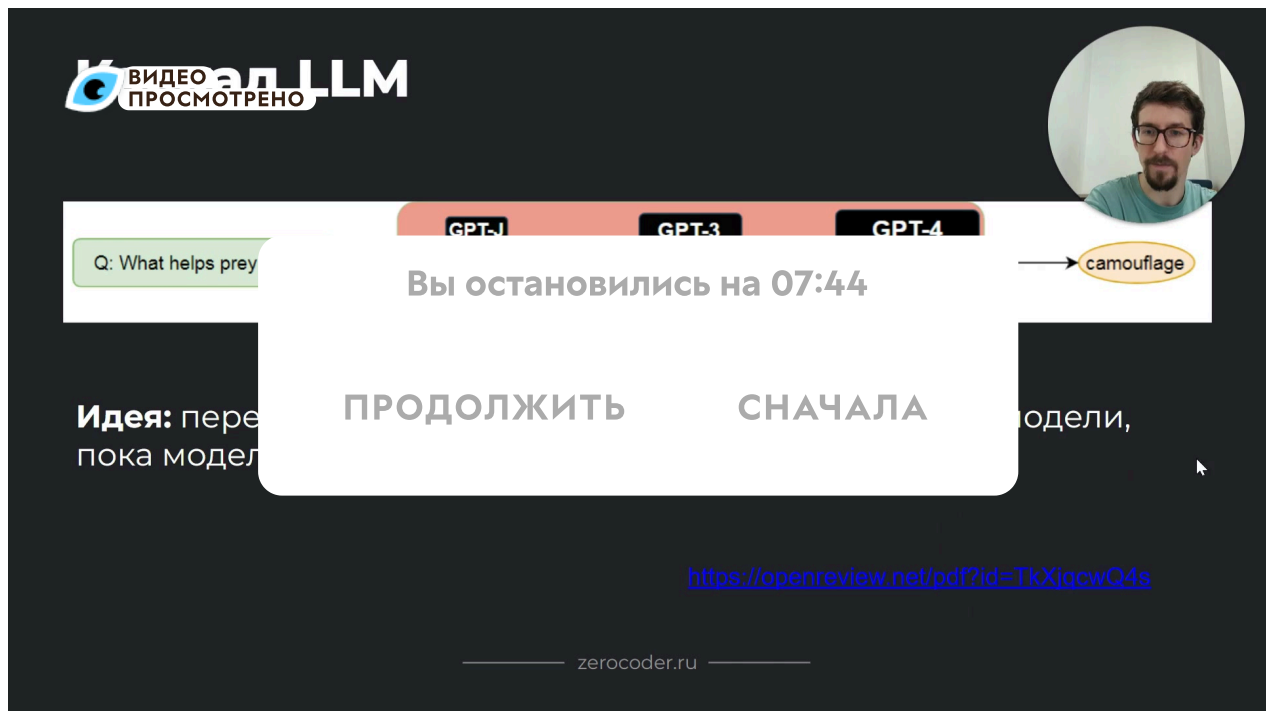


Существует три основных сценария применения мультимодельного подхода:

1. **Разделение задачи на составные части:** Например, при разработке чат-бота для службы поддержки первая модель определяет запрос или намерение пользователя. На основе этого намерения вызывается вторая или третья модель, обрабатывающая запросы различной сложности.
2. **Разные степени сложности задач:** Например, для вопросов-ответов или творческого ассистента можно использовать более сложные или простые модели в зависимости от сложности запроса.

3. **Комбинация ситуаций:** Для суммаризации длинных документов можно разбить документ на части и обрабатывать их разными моделями. Например, для анализа вступления научной статьи достаточно простой модели, тогда как для анализа методов потребуется более сложная модель, способная понимать математическую логику и сложные формулы.

4. Как можно инкрементировать мульти-модельный ИИ

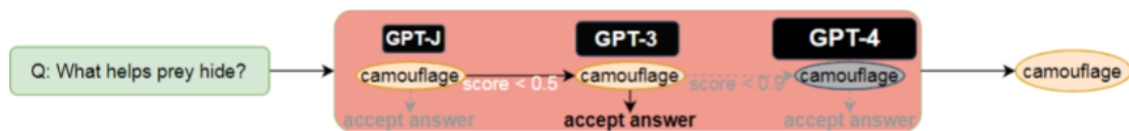


Каскад моделей

Один из подходов к реализации мультимодельного искусственного интеллекта называется **каскад моделей**.

💡 Идея заключается в итеративной проверке ответа с использованием моделей разной мощности.

Сначала запрос передается самой легкой модели, а затем, если уверенность в ответе низка, он передается следующей, более мощной модели. Этот процесс продолжается до тех пор, пока модель не выдаст результат с достаточным уровнем уверенности.



Пример: спросить у простой модели GPT, как хищники охотятся, и она выдаст результат. Для измерения уверенности модели в ответе можно использовать различные подходы.

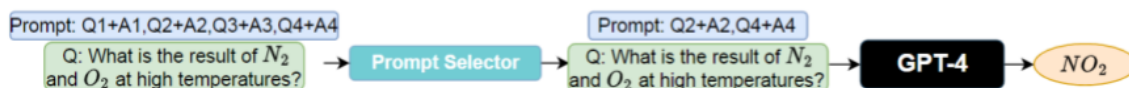
Большинство моделей не возвращают показатель уверенности (confidence score), поэтому нужно разрабатывать собственные правила. Например, можно оценивать ответ по ключевым словам или просить модель указывать, если она не уверена. Если ответ слишком короткий или менее уверенный, запрос передается следующей модели, и так до самой мощной модели, которая уже оценит и выдаст корректный ответ.

Также можно обучить вспомогательную модель для оценки уверенности других моделей. Это требует дополнительных условий и запуска вспомогательной модели локально.

Отбор промптов

💡 Идея: используем вспомогательную модель, чтобы оценить какие части промпта (примеры) наиболее релевантны

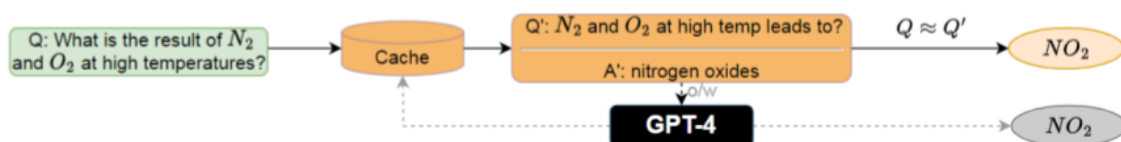
При конструировании промпта можно подавать модели несколько примеров: запрос 1 — ответ 1, запрос 2 — ответ 2 и так далее. Вспомогательная модель выбирает лучшие примеры, которые затем передаются более крупной модели.



Кэширование ответов больших языковых моделей

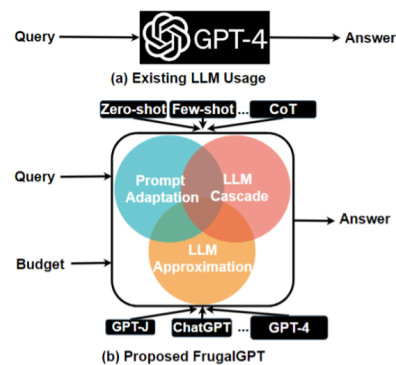
💡 Идея: сохраняем предыдущие вопросы-ответы и если вопрос похож - возвращаем готовый ответ

Если запрос и ответ на него уже были обработаны ранее и сохранены, то повторный запрос не отправляется модели. Вместо этого используется сохраненный ответ. Это особенно полезно для ассистентов, которые часто сталкиваются с повторяющимися запросами. Кэширование позволяет сэкономить ресурсы, так как модель не обрабатывает один и тот же запрос несколько раз.



FrugalGPT

Комбинация вышеописанных методов позволяет сократить стоимость обработки на 50-90% в зависимости от задачи. Это было показано в статье от FrugalGPT. Авторы статьи предлагают корректировать промпт, использовать каскад моделей и кэширование для достижения хороших результатов без значительного снижения качества ответов.

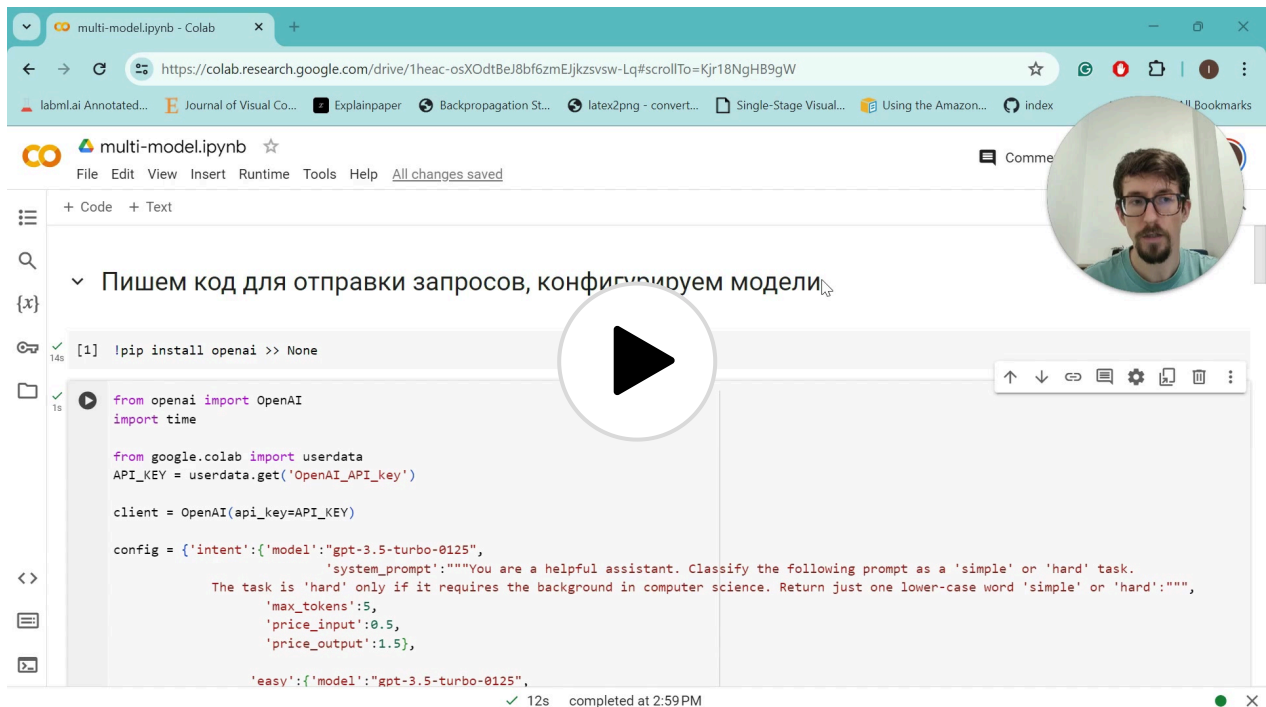


Ссылка на статью: <https://openreview.net/pdf?id=TkXjqcwQ4s>
(<https://openreview.net/pdf?id=TkXjqcwQ4s>)

В более сложных системах можно использовать подходы, описанные в другой статье — <https://arxiv.org/html/2402.01742v1> (<https://arxiv.org/html/2402.01742v1>) . Например, для суммаризации больших документов можно разбивать документ на секции и использовать локальные модели для оценки того, насколько хорошо разные языковые модели справятся с каждой секцией. Это позволяет выбрать наиболее подходящую модель для каждой части документа вместо использования одной мощной модели для всего документа.

Авторы статьи также предлагают кастомные решения, учитывающие ограничения бюджета и скорости работы модели. В идеале можно разработать систему, где указываются предпочитаемые модели и бюджеты, и на основе этих параметров собрать модель, решающую конкретную задачу.

5. Разработка мульти-модельного агента



```
[1] !pip install openai >> None

from openai import OpenAI
import time

from google.colab import userdata
API_KEY = userdata.get('OpenAI_API_key')

client = OpenAI(api_key=API_KEY)

config = {'intent':{'model':"gpt-3.5-turbo-0125",
                  'system_prompt':"You are a helpful assistant. Classify the following prompt as a 'simple' or 'hard' task. The task is 'hard' only if it requires the background in computer science. Return just one lower-case word 'simple' or 'hard':",
                  'max_tokens':5,
                  'price_input':0.5,
                  'price_output':1.5},
          'easy':{'model':"gpt-3.5-turbo-0125",
```

✓ 12s completed at 2:59 PM

❌ Задача: создать ассистента, который будет отвечать на запросы пользователей.

1. Установи библиотеку:

```
!pip install openai >> None
```

2. Импортируй библиотеку и создай клиента:

```
from openai import OpenAI
import time

from google.colab import userdata
API_KEY = userdata.get('OpenAI_API_key')

client = OpenAI(api_key=API_KEY)
```

3. Создай конфигурацию для трёх моделей. Первая модель будет определять сложность запроса. Для этого используем лёгкую модель GPT-3.5 Turbo. Пропиши системный промпт для классификации запросов на "лёгкие" и "сложные". Запросы, требующие знания компьютерных наук, будем считать сложными.

```
config = {'intent':{'model':"gpt-3.5-turbo-0125",
                  'system_prompt':"""You are a helpful assistant.
The task is 'hard' only if it requires the background information.
                  'max_tokens':5,
                  'price_input':0.5,
                  'price_output':1.5}},

         'easy':{'model':"gpt-3.5-turbo-0125",
                  'system_prompt':"You're a helpful assistant.",
                  'max_tokens':None,
                  'price_input':0.5,
                  'price_output':1.5}},
         'hard':{'model':"gpt-4o",
                  'system_prompt':"You're a helpful assistant.",
                  'max_tokens':None,
                  'price_input':5,
                  'price_output':15}}
```

4. Создай функцию для отправки запросов моделям. Она будет принимать тип модели, системный промт и запрос от пользователя, замерять время обработки запроса и рассчитывать стоимость.

5. Добавь функцию классификации запросов и маршрутизацию на соответствующие модели. Классификатор будет определять сложность запроса и перенаправлять его либо на лёгкую модель, либо на сложную.

```

def call_model(prompt, model_type):
    start_time = time.time()
    response = client.chat.completions.create(
        model=config[model_type]['model'],
        messages=[
            {"role": "system",
             "content": config[model_type]['system_prompt']},
            {"role": "user", "content": prompt}
        ],
        max_tokens=config[model_type]['max_tokens']
    )
    latency = time.time() - start_time

    if model_type == 'intent':
        intent = response.choices[0].message.content
    else:
        intent = None
    tokens_input = response.usage.prompt_tokens
    tokens_output = response.usage.completion_tokens
    price = tokens_input*config[model_type]['price_input'] + tokens_
    print(response.choices[0].message.content)
    return price, latency, intent

```

6. Задай тестовые вопросы для проверки работы мультимодельного агента. Например, простой запрос: "Кто лучший актёр, игравший Человека-паука?" и сложный запрос: "В чем разница между оптимизаторами Adam и AdamW?"

```

EASY_QUERY = 'Who is the best actor who played spider-man?'
HARD_QUERY = 'What is the difference between Adam and AdamW optimize

call_model(HARD_QUERY, 'intent')
call_model(EASY_QUERY, 'intent')

```

🔗 Используй только мощную модель и мульти-модельность

Мощная модель

```
price_1,latency_1, _ = call_model(HARD_QUERY, 'hard')
price_2,latency_2, _ = call_model(EASY_QUERY, 'hard')
tot_price = price_1 + price_2
tot_latency = latency_1 + latency_2
print('\n-----\nTOTAL PRICE IS {}, LATENCY {}')
```

Мульти-модельность

```
def router(QUERY):
    price_intent,latency_intent, intent = call_model(QUERY, 'intent')
    if intent.strip().lower() == 'hard':
        price_answer, latency_answer, _ = call_model(QUERY, 'hard')
    else:
        price_answer, latency_answer, _ = call_model(QUERY, 'easy')

    total_price = price_intent + price_answer
    total_latency = latency_intent + latency_answer
    return total_price, total_latency
```

8. Проанализируй результаты работы мультимодельного агента. Сравни стоимость и время выполнения запросов при использовании одной модели для всех запросов и при использовании мультимодельного подхода.

💡 Весь код можно посмотреть здесь:
<https://colab.research.google.com/drive/1heac-osXOdtBeJ8bf6zmEJjkzsvsw-Lq?usp=sharing#scrollTo=sYKCKH3ptXrQ>
(<https://colab.research.google.com/drive/1heac-osXOdtBeJ8bf6zmEJjkzsvsw-Lq?usp=sharing#scrollTo=sYKCKH3ptXrQ>)

Теперь ты видишь, как можно создать мультимодельного агента, который эффективно использует различные модели для обработки запросов разной сложности. Этот подход позволяет экономить ресурсы и повышать качество ответов. Не забывай регулярно анализировать и оптимизировать систему в зависимости от реальных данных и потребностей пользователей.

Основные выводы урока

- мы не можем контролировать токенизацию, но можем уменьшить число используемых токенов;
- можно уменьшать количество используемых токенов простыми методами — разбиение/фильтрация/контроль генерации;
- комбинируя различные модели, можно экономить время/ресурсы без потери качества.

Домашнее задание

1. Повтори шаги, показанные на уроке, и создай своего мультимодельного агента.
2. Проверь, корректно ли модель классифицирует запрос.
3. Попробуй использовать для всех запросов только мощную модель, а затем примени мульти-модельность для обработки запросов. Сравни два полученных результата.
4. В качестве ответа на домашнее задание прикрепи скриншоты ключевых шагов и результатов сравнения.

На следующем занятии

Ты узнаешь про локальное развертывание LLM: обзор эмбедингов, выбор модели и видеокарты, публикация моделей на сервер.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

РЕ9.1 Мульти-модельный ИИ для эффективной работы с ресурсами

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

1. Повтори шаги, показанные на уроке, и создай своего мульти-модельного агента.
2. Проверь, корректно ли модель классифицирует запрос.
3. Попробуй использовать для всех запросов только мощную модель, а затем примени мульти-модельность для обработки запросов. Сравни два полученных результата.
4. В качестве ответа на домашнее задание прикрепи скриншоты ключевых шагов и результатов сравнения.



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Следующий урок